

App Features (Functional – no crypto yet)

Think of this as your **basic working system**.

1. User Authentication

- User registration (name, email, password)
- User login/logout
- Profile view & edit

2. Academic Feed (Posts)

- Create post (text / notes)
- View all posts (feed)
- Edit own posts
- Delete own posts

3. Messaging System

- Send message to another user
- View chat history
- Simple one-to-one chat

4. Document Vault

- Upload files (PDF, images, etc.)
- View uploaded files
- Delete files

5. Anonymous Reporting

- Submit a report

- Admin can view reports
- (Initially not anonymous - just normal submission)

6. Role-Based Access

- Admin dashboard:
 - View users
 - View reports
- User dashboard:
 - Posts, messages, files

7. Session Management (basic)

- Login creates session/token
- Logout destroys session

How to Build the App (Step-by-Step)

STEP 1: Setup Project

Tech Stack (same as your previous projects)

- Frontend → React.js
- Backend → Node.js + Express
- Database → MySQL

Basic Folder Structure

```
ciphercampus/
├── client/      (React frontend)
├── server/      (Node backend)
│   ├── routes/
│   ├── controllers/
│   ├── models/
│   └── config/
└── database/
```

STEP 2: Database Design (IMPORTANT FIRST)

Create these tables:

Users Table

- id
- username
- email
- password
- role (admin/user)

Posts Table

- id
- user_id
- content
- created_at

Messages Table

- id
- sender_id
- receiver_id
- message

- timestamp

Documents Table

- id
- user_id
- file_path
- uploaded_at

Reports Table

- id
- user_id
- report_text
- created_at

STEP 3: Build Authentication

Backend:

- Register API
- Login API

Frontend:

- Signup page
- Login page

At this stage:

- Store password normally (later we hash)

STEP 4: Dashboard System

After login:

- Redirect user to dashboard

Pages:

- Home (feed)
- Messages
- Documents
- Reports
- Profile

STEP 5: Posts Module

Backend:

- Create post API
- Get all posts API
- Update post API
- Delete post API

Frontend:

- Post form
- Feed display

STEP 6: Messaging System

Backend:

- Send message API
- Get messages API

Frontend:

- Chat UI (simple list)

STEP 7: Document Upload

Backend:

- Upload file (use multer)
- Store file path in DB

Frontend:

- Upload button
- File list

STEP 8: Reporting System

Backend:

- Submit report API
- Admin: view reports API

Frontend:

- Report form
- Admin dashboard

STEP 9: Role-Based Access

Add:

- role = "admin" or "user"

Logic:

- Only admin can:
 - View reports
 - Manage users

STEP 10: Basic Session Handling

- Use JWT or sessions
- Protect routes:
 - Only logged-in users can access

